



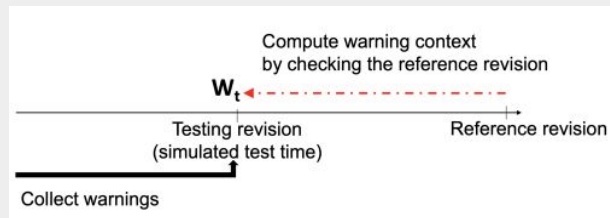
Centro de
Informática
UFPE

Detecting False Alarms from Automatic Static Analysis Tools: How Far are We?

Reprodução e Experimentos Adicionais

Artigo

Problema e Experimentos



Fonte: Kang et al. (2022)

Contexto e Problema

“Golden Features”

- ASATs (Automated Static Analysis Tools) são amplamente utilizadas para detectar bugs com baixo custos, mas sofrem com altas taxas de alarmes falsos
- Pesquisas anteriores propuseram o uso de aprendizado de máquina e um conjunto de 23 “Golden Features” para classificar avisos das ASATs como acionáveis (bugs reais) ou não e relataram um desempenho quase perfeito

warning context in method	comment-code ratio	warning pattern	file creation revision	LOC added in package (past 3 months)
warning context in file	method depth	warning type	developers	average lifetime for warning type
warning context for warning type	file depth	warning priority	parameter signature	warnings in package
defect likelihood for warning pattern	methods in file	package	method visibility	
discretization of defect likelihood	classes in package	file age	LOC added in file (last 25 revisions)	

Desempenho quase perfeito?

Problemas no experimento original

- O estudo identificou dois problemas principais nos experimentos realizados anteriormente:
 - **Data Leakage:** algumas das features estavam inadvertidamente “vazando” a resposta correta pro modelo
 - **Data Duplication:** os conjuntos de treinamento e teste do modelo tinham muitos avisos em comum
- Além disso, o trabalho questionou a robustez da heurística de ***closed-warning**** utilizada para construir o ***ground truth***
 - Realizaram experimentos com revisões de referências cada vez mais no futuro

* Um aviso é considerado acionável se não está mais aberto na revisão de referência

Data Leakage

Features com vazamento

- Cinco das “Golden Features” eram computadas a partir de uma informação extraída da revisão de referência, cronologicamente no futuro
 - Em um cenário real, isto é impossível
- Além disso, a revisão de referência é utilizada para computar o **ground truth**
 - Utilizá-la para computar as features acabava também “vazando” a resposta correta para o modelo
- O estudo fez experimentos com:
 - Um modelo que usava apenas as features com vazamento ✓
 - Um modelo sem as features com vazamento ✗
 - Um modelo com as features problemáticas re-implementadas ✗

Data Duplication

Entre conjuntos de treinamento e teste

- O estudo encontrou um alto número de avisos repetidos entre os conjuntos de treinamento e teste usados pelos modelos
- Isso aumenta o desempenho dos modelos, pois eles podem “lembrar” as classificações computadas no conjunto de treinamento ao executar para o conjunto de testes
- O estudo fez experimentos com:
 - Um modelo *dummy* que verifica se já computou a classificação para este aviso antes e reporta o mesmo resultado ✓
 - Conjuntos de treinamento e teste sem duplicações ✗

Heurística de closed-warning

Diferentes revisões de referência

- O estudo questiona a robustez dessa heurística como **ground truth**
- Hipotetizam que quanto mais a frente a revisão de referência, mais avisos seriam marcados como acionáveis
- Fazem experimentos com revisões de referência:
 - 2 anos a frente
 - 3 anos a frente ↑
 - 4 anos a frente ↑
- Fazem uma classificação manual dos avisos
 - Verificaram que muitos avisos são fechados indiretamente
 - Apenas 47% dos avisos fechados foram considerados acionáveis por humanos

Reprodução

*Mesmos Experimentos
+ SpotBugs*

Mesmos Experimentos

Comparação dos resultados

- Como um primeiro passo, utilizamos os artefatos disponibilizados pelos autores para re-executar os experimentos
 - Fizemos os ajustes necessários para conseguir executar os scripts
 - Adicionamos novos artefatos para automatizar a geração das tabelas
- Comparamos os resultados obtidos na nossa execução com os reportados pelo artigo
 - Procuramos por erros e inconsistências nas tabelas do artigo

Mesmos Experimentos

Após solução dos problemas

Technique	Precision	Recall	F1
Golden Features SVM	0.84	0.95 (0.94)	0.88
- leaked features	0.29 (0.26)	0.70	0.37 (0.38)
- data duplication	0.86 (0.88)	0.94 (0.93)	0.89 (0.90)
- data duplication and leaked features	0.26 (0.27)	0.56 (0.57)	0.31
+ reimplemented leaked features	0.32	0.57	0.39 (0.38)
Golden Features kNN (with $k = 10$)	0.92 (0.91)	0.57	0.68
Golden Features kNN (with $k = 5$)	0.86	0.72	0.78
Golden Features kNN (with $k = 3$)	0.88 (0.87)	0.78	0.83 (0.82)
Golden Features kNN (with $k = 1$)	0.85 (0.87)	0.90	0.87 (0.84)
Only leaked features SVM	0.79	0.94	0.83

Mesmos Experimentos

Diferentes revisões de referência

Project	2 years			3 years			4 years		
	F1	F1 (dummy)	AUC	F1	F1 (dummy)	AUC	F1	F1 (dummy)	AUC
<i>ant</i>	0.00	0.38	0.43	0.13	0.60	0.48	0.00	0.60	0.32
<i>cassandra</i>	0.56	0.59	0.52	0.61	0.63	0.41	0.58	0.60	0.48
<i>commons</i>	1.00	0.67	1.00	1.00	0.67	1.00	1.00	0.67	1.00
<i>derby</i>	0.06	0.18	0.33	0.58	0.74	0.46	0.72	0.80	0.52
<i>jmeter</i>	0.13	0.16	0.58	0.12	0.27	0.40	0.71	0.95	0.52
<i>lucene-solr</i>	0.56	0.62	0.58	0.58	0.66	0.57	0.63	0.80	0.53
<i>maven</i>	0.25	0.29	0.41	0.27	0.28	0.44	0.27	0.28	0.44
<i>tomcat</i>	0.54	0.59	0.51	0.51	0.57	0.47	0.64	0.69	0.54
Average	0.39	0.43	0.54	0.47 (0.48)	0.55	0.53	0.57	0.67	0.54

Mesmos Experimentos

Classificação Manual

Dataset	F1	F1 (dummy)	AUC
<i>Original dataset</i>	0.39	0.43	0.54
<i>- unconfirmed actionable warnings</i>	0.39 (0.61)	0.59 (0.57)	0.50 (0.66)
<i>Projects using Findbugs</i>	0.36 (0.43)	0.42 (0.44)	0.64 (0.62)
<i>- unconfirmed false alarms</i>	0.42 (0.41)	0.43 (0.46)	0.63 (0.60)

SpotBugs

Sucessor do FindBugs

- Executamos SpotBugs para 6 revisões dos últimos 7 anos de 7 dos 9 projetos originais do artigo
- Contamos o número de avisos que permaneciam abertos em cada par ordenado de revisões de cada projeto
- Observamos a mesma tendência de aumento no número de avisos acionáveis

Project	2 years	3 years	4 years
<i>ant</i>	99,82%	99,19%	98,56%
<i>cassandra</i>	60,51%	50,89%	45,39%
<i>commons-lang</i>	95,77%	87,32%	85,92%
<i>derby</i>	98,09%	96,36%	89,96%
<i>maven</i>	46,53%	45,10%	44,09%
<i>phoenix</i>	89,71%	83,74%	73,22%
<i>tomcat</i>	94,12%	91,97%	87,52%
Average	83,51%	79,22%	74,95%

Análises manuais

Conferência com filtros
FindBugs

validação manual de alertas fechados

Alertas FindBugs fechados pela heurística do estudo

Entrada da replicação: Alertas fechados pela heurística

Casos em que o FindBugs reportava um alerta em uma revisão anterior, mas ele não aparecia na revisão futura de referência, com o arquivo ainda existente.

Procedimento manual: Como cada alerta foi julgado



Resultados obtidos

Alertas FindBugs fechados pela heurística do estudo

Distribuição dos rótulos:

- **Acionável:** 660 (48,7%)
 - Mudança direcionada ao problema reportado pelo FindBugs
- **Falso Alarme:** 176 (13,0%)
 - Código original parecia comportamento esperado, ou a mudança apenas removeu/silenciou um alerta que não era bug real.
- **Desconhecido:** 520 (38,3%)
 - Refatoração ampla, funcionalidade removida/reescrita, método movido/renomeado ou código gerado.

A heurística “alerta fechado = bug corrigido” superestima alertas acionáveis

Leitura dos resultados

- Muitos alertas somem por refatoração ou mudança funcional, não por correção do bug pattern.
- 51,3% dos alertas “fechados” não foram confirmados como correções acionáveis.
- Impacto: modelos treinados com esses rótulos podem aprender um oráculo imperfeito e superestimar desempenho.

Lições Aprendidas

Lições Aprendidas

- Questionar a validade de um experimento que apresenta resultados quase perfeitos
- Fazer uso de análise manual para entender resultados inesperados
- Elaborar artefatos de pesquisa pensando em reprodutibilidade em ambientes diversos

Trabalhos Relacionados

Trabalhos Relacionados

- *Is There A “Golden” Feature Set for Static Warning Identification?*
 - Propõe o conjunto de 23 “Golden Features” para classificação de avisos
 - Mostrou desempenho quase perfeito de modelos que usam essas features
- *An Empirical Study of False Negatives and Positives of Static Code Analyzers From the Perspective of Historical Issues*
 - Analisou 350 avisos históricos de ASATs (incluindo SpotBugs) para identificar as principais causas de falsos positivos e negativos reportados por desenvolvedores
 - Identificou alguns problemas, como falta de contexto semântico por parte das ferramentas e dificuldade de modelar cenários reais
- *An Empirical Study on Bug Severity Estimation using Source Code Metrics and Static Analysis*
 - Avaliou a eficácia das ASATs em prever a severidade de bugs
 - Mostrou que seus avisos isoladamente são fracos preditores

Referências

KANG, H. J.; AW, K. L.; LO, D. Detecting False Alarms from Automatic Static Analysis Tools: How Far Are We? ICSE 2022.

WANG, J.; WANG, S.; WANG, Q. *Is there a "golden" feature set for static warning identification? An experimental evaluation.* ESEM 2018.

CUI, Y. et al. *An Empirical Study of False Negatives and Positives of Static Code Analyzers From the Perspective of Historical Issues.* ICSE 2025.

MASHHADI, E. et al. An Empirical Study on Bug Severity Estimation using Source Code Metrics and Static Analysis. Journal of Systems and Software, 2024.



Centro de
Informática
UFPE